

わかりやすい説明の組み立て方

 GIZUMO



Outline

- コードを説明する意義
- 説明文を分解する



説明をロジカルに組み立てる手順

ではここでは、greetという関数を例として実際に説明を組み立てていきましょう。

説明の組み立ては以下の手順で行います。

1. まずは思いついた説明を書いてみる
2. 1.の説明の中から **最も伝えたい部分** を取り出す
3. 2.の説明の後に、具体的な処理の内容を付け足す
4. 3.の説明の後に、なぜその処理を行なっているかを付け足す

1.まずは思いついた説明を書いてみる

```
function greet(name, age) {  
  let greetText = '名前は' + name + 'です。年齢は' + age + '歳です。';  
  return greetText;  
}  
  
console.log(greet("鈴木", "26"))
```

この関数は、nameとageを引数として受け取って、「名前は」「です。年齢は」「歳です。」の文字列と引数を結合したgreetTextという変数を戻り値として返す関数です。
戻り値として受け取った値をコンソールに表示しています。

1.まずは思いついた説明を書いてみる

この関数は、nameとageを引数として受け取って、「名前は」「です。年齢は」「歳です。」の文字列と引数を結合したgreetTextという変数を返り値として返す関数です。返り値として受け取った値をコンソールに表示しています。

思いついた限りで説明を自分でも書いてみましょう。

上記のように、少し説明としてわかりづらいな？と感じた場合は、次に説明を要素ごとに分解していきます。

2.最も伝えたい部分を取り出す

コードを説明する際には

- コードの概要(一番伝えたいこと)
- コードの中の具体的な処理
- なぜその処理を行なっているのか

この3点の順番で意識していくと非常に相手に伝わりやすくなります。

まずは**一番伝えたいこと**を抜き出していきましょう。

2.最も伝えたい部分を取り出す

この関数は、nameとageを引数として受け取って、「名前は」「です。年齢は」「歳です。」の文字列と引数を結合したgreetTextという変数を返り値として返す関数です。
返り値として受け取った値をコンソールに表示しています。

この説明の中で最も伝えたい部分は引数として受け取った値を「表示」するのではなく、あくまで返り値として返しているだけだということです。

この関数は何をしているかを説明する際には、「何を」の部分に注目して、それを1文で書ける範囲で探してみましょう。

2-1.結論を一番最初に持ってくる

この関数は〇〇をする関数です。

一番伝えたいことを切り取ったら、これを **説明の一番最初に持ってきてみましょう。(アンサーファースト)**

この結論が説明の後半に回ってしまうと、聞き手は集中をずっと続けなくてはならず、うまく話を概要をつかめなくなってしまう。

3.どんな処理をしているのかを説明する

この関数は〇〇をする関数です。

実際の処理の流れをコードに沿って説明します。

let greetTextでgreetTextという変数を定義し、代入演算子を使用して引数と文字列を結合した値を代入します。

その後、returnを使用し、関数の呼び出し元に返り値としてgreetTextを返します。

では、結論を伝えた後に、やっとコードの中身まで説明していきましょう。

一番最初に結論を伝えているため、聞き手はなんの話をしているかをしっかりと理解した上で話を聞くことができます。

4.なぜその処理をしているのかを説明する

この関数は〇〇をする関数です。

実際の処理の流れをコードに沿って説明します。

let greetTextでgreetTextという変数を定義し、代入演算子を使用して引数と文字列を結合した値を代入します。

その後、returnを使用し、関数の呼び出し元に返回值として greetTextを返します。

返ってきた値を console.logの引数に渡すことにより、コンソールに表示させています。

最後に、何故その処理を行っているのかを説明します。

一部のコードを説明する際には、何故その処理が必要なのかを伝えなければ、**聞き手はその処理が適切かどうか判定することはできません。**

コードを説明する際には、必ず背景も説明してあげましょう

説明を上手に行う心構え

1. **最も伝えたい部分** を一番最初に話す
2. 結論を最初に持ってくることで続く説明がわかりやすくなる
3. コードを説明するときには必ずその **背景** も説明する